Software Specification

By Jim Truong, Giovanna Dunker Estruquel, Ben Choi, Hamza Vaid, Shogo Stuck

Table of Contents

Glossary **3**
1. Software Architecture Overview **5**
1.1. Main data types and structures 5
1.3. Module interfaces 6
1.4. Overall program control flow 6
2. Installation **7**
2.1. System requirements and compatibility 7
2.2. Setup and configuration 7
2.3. Building, compilation, and installation 7
3. Documentation of Packages, Modules, and Interfaces **8**
3.1. Detailed description of data structures 8
3.2. Detailed description of functions and parameters 9
3.3. Detailed description of input and output formats 10
4. Development Plan and Timeline **11**
4.1. Partitioning of tasks 11
4.2. Team member responsibilities 12
5. Back Matter **12**
Copyright 12
Error messages 12
References 15
Index 16

Glossary

- **Ant (Pawn)** – Can move forward one square (or two squares on its first move) and captures diagonally.
- **Anteater** – A special piece that can move one square in any direction (like a king), but can only capture ants (pawns). When capturing, the anteater can perform ant eating, meaning it can eat multiple adjacent ants in a single move (along the same row or column), ending on the position of the last ant eaten.
- **Ant Eating** – A special move where the anteater captures one or more adjacent ants in a sequence during a single turn. The anteater continues eating ants that are next to each other in the same row or column and finishes on the last square chosen by the player.
- **Bishop** – Can move any number of squares diagonally.
- **Board** – A data structure representing the game state as an 8x10 grid of squares, each holding a piece or empty, used to track and update gameplay.
- **Black (Player)** – One of the two sides in the game. The black player typically moves second and controls the black-colored pieces.
- **Capture** – One piece attacks another, resulting in the removal of the attacked piece from the board.
- **Castling** – The king may move two pieces towards the rook, and the rook moves to the opposite side of the king, so long as:
    1. Neither the king nor the rook involved in the castle has moved in this game
    2. There are no pieces between the king and the rook involved in the castle
    3. The king is not in check at the time of the castle
    4. The king is not moving through any square in which it would be checked
- **Check** – When the king is under attack but has a legal move to escape.
- **Checkmate** – When the king cannot escape danger, and the game is over.
- **Computer (Player)** – The automated opponent controlled by the program, making moves based on implemented game logic and rules.
- **En Passant** – A special rule that allows the acting player to capture an ant (pawn) that just moved two squares forward on the square behind it using another pawn. The rule was added to make it more difficult to pass pawns on adjacent files.
- **File** – Synonymous with “columns,” which are labeled A - J

- **GUI** – Short for “graphical user interface,” it’s what the user sees and interacts with in the final implementation of Anteater Chess.
- **Human Player** – The user interacting with the program who inputs moves and controls one side, either white or black.
- **King** – The most important piece; moves one square in any direction and must not be checkmated.
- **Knight** – Moves in an L-shape and is the only piece that can jump over others.
- **Piece** – A struct that occupies each square of the board and specifies a piece’s type, color, and position.
- **PieceType** – A property of the Piece struct that keeps track of what type of piece occupies a square, such as knight, king, queen, or bishop (not an exhaustive list).
- **Promotion** – When an ant reaches the last rank, it can be promoted to any piece (not a pawn).
- **Position** – A property of the Piece struct that keeps track of the square on the board a particular piece occupies
- **Queen** – Can move any number of squares horizontally, vertically, or diagonally.
- **Rank** – Synonymous with “rows,” which are labeled 1 - 8.
- **Raylib** – A C programming library that allows for the creation of GUI elements and the processing of user mouse inputs.
- **Rook** – Can move any number of squares horizontally and vertically.
- **Stalemate -** The acting player has no legal moves and is not in check, resulting in a draw
- **White (Player)** – One of the two sides in the game. The white player moves first and controls the white-colored pieces.

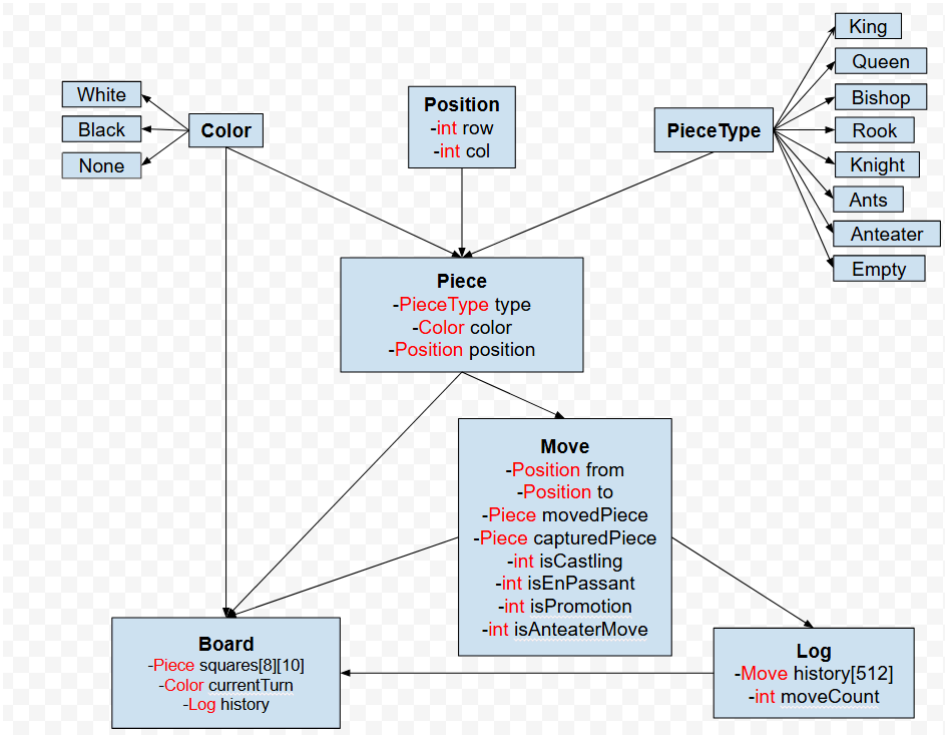
1. Software Architecture Overview

1.1. Main data types and structures

The core data structure of the Anteater Chess program is a two-dimensional array that represents the game board. The board consists of 8 rows and 10 columns, matching the modified chess layout described in the user manual. Each position on the board stores a piece object that contains important information such as the type of piece (ant, anteater, bishop, etc.), the color of the piece (white or black), and whether the piece has moved before. This information is necessary for methods such as castling and en passant. In addition to the board, a move structure is used internally to represent a player's move, storing the starting and ending positions of a piece. These data structures allow the program to efficiently track and update the state of the game throughout execution.

Our goal is to structure the game according to object-oriented programming (OOP) principles. Using enumerations (`enum`) allows us to group all the valid piece types and colors into one place, which we can use later on in a structure (`struct`) for Piece. Additionally, it's a necessity that we also include an "empty" piece type to fill squares on the board that are not occupied.

1.2. Major software components



The structure of the board considers each square on the board an object where the PieceType and Color of the occupied (or non-occupied) square changes based on what is validated by the Move component. The different components are encapsulated so as to communicate with one another while maintaining separation of duties, and making it easier to troubleshoot by isolating bugs to their respective components rather than tackling the entire codebase at once.

One consideration we made was the actual structure of the Board, which is a 2D array of the Piece object, but we also considered different board representations. There are other ways to represent the board, such as by integer or bit representation, or a one-dimensional array, to simplify the number of searches and loops necessary to traverse the game's information. We settled on a 2D array because it vastly simplifies the codebase on the level of readability, and it's very important that members of the team can adapt to changes we make. It's also possible to refactor the code to something like a one-dimensional array if time permits, so that our computer engine will be able to calculate lines faster.

1.3. Module interfaces

Universal Chess Interface (UCI) - A text protocol that enables chess engines to interact with the graphical user interfaces (GUIs):

- Functions such as `displayBoard(), setTime(), notifyCheck(),` and `displayWinner().`
    - The actual game mechanics underlying these visuals will be computed elsewhere, these functions simply update what the user sees
- Functions such as `IsMouseButtonPressed(), GetMousePosition(),` and `InitWindow().`
    - These are from the raylib library and allow us to take user input in the form of buttons and mouse clicks.

Game Interfaces - These allow communication between the game's core modules

- Control flow through functions like `initBoard()`
- Information gathering through functions like `getPieceAt()` and `isInCheck()`
- Engine turn computations through functions like `evaluateBoard()`
- These game interfaces are described in greater detail in section 3.3 of this document

1.4. Overall program control flow

- Board display
- Prompt the User to play against a computer or another player

- If the user picks to play against a computer, prompt the user to select a side
- Begin of gameplay loop
    - Prompt the user to make a move
    - Check the user's move if it is valid
    - Check if the move is legal
    - Execute the move and update the board
    - Switch to the next color
    - Loop
        - Loop ends when the game is ended via checkmate, stalemate, or when the game is terminated

2. Installation

2.1. System requirements and compatibility

- Operating System: Windows / macOS / Linux
- Compiler: GCC (C language)
- Memory: TBD
- Storage: TBD

2.2. Setup and configuration

Currently, the program has not yet been developed, so a complete setup and configuration process is not available. However, the expected installation procedure is as follows:

1. Download and extract the program files.
2. Open terminal and navigate to project directory.
3. Compile the program using the command:

   `make`
4. Run the program using:

   `./chess`

2.3. Building, compilation, and installation

- Delete the project folder
- Remove compiled files if needed.

  (TBD) Depending on the file specifications and added features.

3. Documentation of Packages, Modules, and Interfaces

3.1. Detailed description of data structures

The Anteater Chess program will be organized into several core modules that work together to represent the game state, validate moves, display the board, and record the gameplay. We will be using an object oriented approach to create all the data structures used in the project including but not limited to:

1. `PieceType`: Represents any kind of chess piece such as king, queen, rock, bishop, knight, ant, and most importantly anteater.

```
typedef enum {
    KING, QUEEN, ROOK, BISHOP, KNIGHT, PAWN, ANTEATER
} PieceType;
```

2. `Color`: This will represent the owner of the piece. The color can either be white or black and will be used to determine turn order, legal movement direction, and legal checks.

```
typedef enum {
    WHITE, BLACK, NONE
} Color;
```

3. `Position`: This will store the current board location using rows and column coordinates. Internally, the program may use integer indexes which can be translated to rank and file notation for the user.

```
typedef struct {
    int row;
    int col;
} Position;
```

4. `Piece`: This will combine both `PieceType`, `Color`, `Position`, and many more to determine possible legal moves and outcomes.

```
typedef struct {
    PieceType type;
    Color color;
    Position position;
} Piece;
```

5. `Move:` This structure will store all current information for one move. It will contain starting position and ending position and additional information for the chess program such as the piece information and if it can promote, castle, en passant, or be a part of a special anteater move.

```
typedef struct {
    Position from;
    Position to;
    Piece movedPiece;
    Piece capturedPiece;
    int isCastling;
    int isEnPassant;
    int isPromotion;
    int isAnteaterMove;
} Move;
```

6. `Board`: This will store the current full board as a two-dimensional array of squares containing either a piece or an empty value such as NULL. The board will also store additional gamestate information such as current turn and move-history if needed.

```
typedef struct {
    Piece squares[8][10];
    Color currentTurn;
    Move history[512];
    Int moveCount;
} Board;
```

3.2. Detailed description of functions and parameters

We will divide the program into modules, and each module will expose functions through header files. These functions serve as interfaces between packages.

1. **Board Module (Includes GUI)**

   This module handles the game state, piece placement, and GUI creation.

   ```
   void initBoard(Board *board);
   ```

   Initializes the board to start the board construction and to initialize the game

   ```
   Piece getPieceAt(Board *board, Position pos);
   ```

   Returns a piece located at a given position.

   ```
   void setPieceAt(Board *board, Position pos, Piece piece);
   void movePiece(Board *board, Position from, Position to);
   ```

   Sets a piece (empties in the case of castling or en passant, replaces in the case of regular captures) or moves a piece to a certain position on the board.

2. **Rules Module**

This module determines whether moves are valid according to the rules of the game.

```
int isValidPosition(Position pos);
```

Checks whether a position is inside the board boundaries.

```
int isLegalMove(Board *board, Move move, Color player);
```

Checks whether a move is legal for the specified player.

```
int isInCheck(Board *board, Color player);
int isCheckmate(Board *board, Color player);
```

Determines whether the given player's king is currently in check or checkmate.

3. **Log Module**

   This module records moves to a text file or on the screen

```
void logMove(FILE *fp, Move move, int turnNumber);
void saveGameLog(const char *filename, Log *log);
```

   Write moves to the log file and the complete move history in a file

4. **Automatic Module**

   This module is responsible for the automatic computer moves

```
Move chooseBestMove(Board *board, Color player);
int evaluateBoard(Board *board, Color player);
```

   Selects the moves for the computer player and a score to the current board state.

3.3. Detailed description of input and output formats

The program will use a graphical user interface (GUI) for move input and board display.

-

The input to the program will be a move selected through the GUI. A move is entered in two steps. First, the player will click on the square containing the piece they want to move. Second, the player will click on the destination square where they want the piece to go.
After the second click, the program will interpret the action and the software will check if the move is legal and if the move is illegal, the move is rejected and an error message is displayed.

- **Output Format**
  The primary output of the program is the updated GUI board display. After each valid move, the interface visually updates the location of the pieces. The GUI may also highlight the current selected piece, possible legal destinations, invalid move attempts, and check or checkmate conditions. Additional output will be shown in the terminal to show status messages such as illegal moves and checks.
- **Log File Output Format**
  The program will still store each move in a human-readable log file. Each recorded move includes the turn number, player color, and board positions involved in the move.
  Example:
  Turn 1: White F2 F4
  Turn 1: Black E7 E5

4. Development Plan and Timeline

4.1. Partitioning of tasks

The development of Anteater Chess is divided into several key components to ensure that all major features are implemented. The first stage focuses on core functionality, including board creation, piece structures, and movement logic. This includes implementing the board as a two-dimensional array, defining piece types, and coding individual movement rules. The next stage involves developing the move validation system, which ensures that all moves follow the rules of chess and the anteater extension, including handling captures, checks, checkmates, and stalemates.

After the core gameplay is functional, additional features such as move logging and overall control flow will be implemented. The graphical user interface (GUI) or ASCII interface will also be developed to improve user interaction, including displaying the board and highlighting legal moves. Finally, the computer engine will be implemented, starting with randomly selected legal moves and later expanded to more advanced decision-making algorithms.

4.2. Team member responsibilities

Each team member is assigned specific tasks based on different components of the system to allow for efficient integration:

- Hamza Vaid:
  - Assist in implementing the Rules module
  - Implement board and piece classes/structures and piece logic (movement rules)
- Jim Truong:
  - Assist in implementing the class structure
  - Assist in implementing the Log module
- Shogo Stuck:
  - Legal move checking
  - Integrate checks, checkmates, and stalemates
- Ben Choi:
  - Create scaffolding code for basic program structure
  - Capture logic for pieces and edge cases, such as En Passant, castling, and boundary checking
- Giovanna Dunker Estruquel:
  - Develop GUI
  - Implement user interaction features, such as displaying the board
  - Help integrate control flow (start game, exit game, game mode selection)
- Matthew:
  - Start implementation on the Automatic Module

5. Back Matter

Copyright

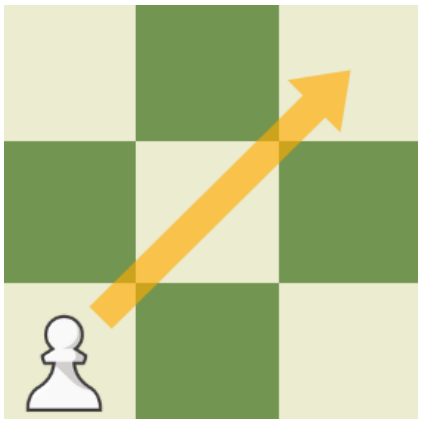
© 2026 Anteater Chess Team (En Passant Police). All rights reserved.

This software was developed as part of the EECS 22L course at the University of California, Irvine and is intended for educational purposes only.

Error messages

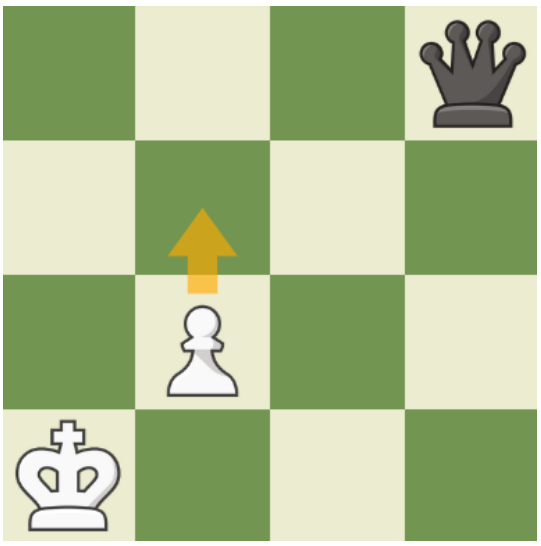
The Anteater Chess program provides feedback to the user when an invalid input or illegal action happens. Most of these will be rule violations pertaining to piece movement or king checks, but there are also other errors stemming from how the application runs. The following are error messages that will be implemented:

Invalid Move - A move that is inconsistent with how the pieces are allowed to act:

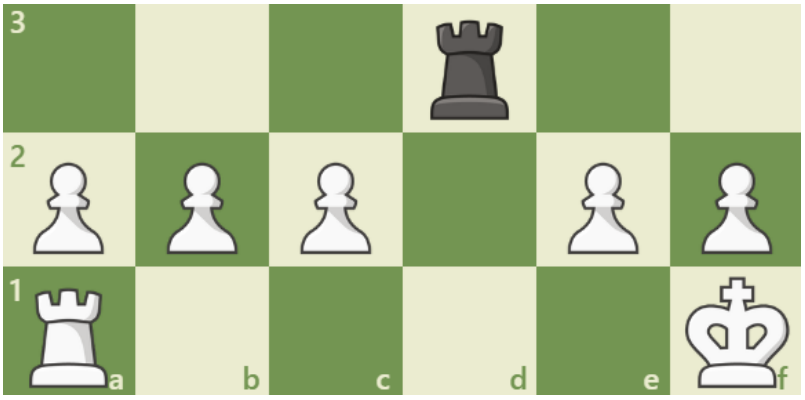


This move is not allowed because a pawn may only move one or two spaces forward, with the exception of captures which only allows one square of diagonal movement, not two.

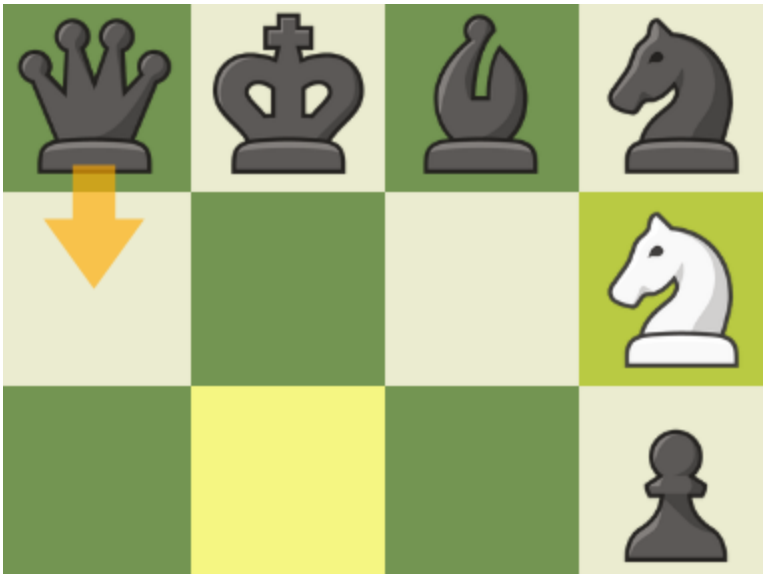
Illegal Move - A move consistent with piece movement, but not allowed due to checks



While ants are typically allowed to advance one square forward, white cannot play that move as it would place their king in check.



White would typically be allowed to castle here, but since the king would move through a square in which the rook checks the king, it is an illegal move.



In this position, the black king is in check, so moving the queen is illegal. This is because black must escape check by capturing the knight or moving the king.

Wrong Player Move - The same player can never play two moves in a row, so if a player attempts this, they will be met with a warning message

Move Out of Bounds - If a player specifies that they want to move a piece to or from a square outside the legal grid, lets say, F12 or Z3, they will be met with a warning message.

Missing File Error - If one of the game's files is missing, impeding the playability of the game, the application will provide an error message and the names of the missing files.

References

(TBD) As we go through this project, we will add more references for different areas of the project.

Course / Project Materials:
Dömer, R. (2026). *EECS 22L: Project 1 – Anteater Chess Specification*. University of California, Irvine.
Yuan, C. (2026). *Chess Software Architecture Specification Grading Criteria*. University of California, Irvine.

Chess Rules:
Chess.com. *How to Play Chess: Rules for Beginners*.
FIDE. (2023). *FIDE Laws of Chess.*

Graphical Interface:
"Raylib - Cheatsheet." *Raylib*, www.raylib.com/cheatsheet/cheatsheet.html.

Index

A
Ant – 3
Anteater – 3
Ant Eating – 3

B
Bishop – 3
Black – 3, 6
Board – 7, 8
Board Module – 9

C
Capture – 3
Castling – 3
Check – 3
Checkmate – 3
Computer (Player) – 3

D
Download – 6

E
En Passant – 3
En Passant Police – 12
Enum – 5

F
File (Column) – 3
File (Data) – 6, 7, 10

G
GUI – 10

I
Illegal Move – 6, 12
Input Format – 10
Installation – 6
Invalid Move – 6, 12

K
King – 4
Knight – 4

L
Log Module – 9

M
Move – 8

O
Operating System – 6
Output Format – 10

P
Pawn (Ant) – 3
Piece – 7, 8
Position – 7
Promotion – 3

Q
Queen – 4

R
Rank – 4, 7
References – 14
Rules Module – 9
Rook – 4

S
Stalemate – 4, 7
Struct – 5

U
UCI (Universal Chess Interface) – 6

W
White – 4, 5, 8